Lab Mid Exam: HOG vs SIFT on Blood-Cell Images

Name: Muhammad Azeen Waqas Reg No: 012

Seed: 12 | HOG cell: 8×8 (last digit even) | SIFT grid: 4×4 (last digit 0–4)

CODE CELL 1

```
!pip install -q medmnist
import numpy as np, cv2, medmnist
from medmnist import BloodMNIST
import matplotlib.pyplot as plt
from skimage.transform import rotate
from skimage.feature import hog
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix

SEED = 12
HOG_CELL = (8, 8)
SIFT_GRID = 4
CLASSES = [1, 5, 6, 7]
NAMES = {1: 'eosinophil', 5: 'monocyte', 6: 'neutrophil', 7: 'platelet'}
```

Q1. Data setup

CODE CELL 3

```
def load(split, n):
    ds = BloodMNIST(split=split, size=64, download=True)
    labels = ds.labels.flatten()
    rng = np.random.default_rng(SEED)
    X, y = [], []
    for c in CLASSES:
        idx = rng.choice(np.where(labels == c)[0], n, replace=False)
        for i in idx:
            g = cv2.cvtColor(ds.imgs[i], cv2.COLOR_RGB2GRAY)
            X.append(cv2.resize(g, (128, 128), interpolation=cv2.INTER_LINEAR))
            y.append(c)
    return np.array(X), np.array(y)

Xtr, ytr = load('train', 100)
Xva, yva = load('val', 30)
Xte, yte = load('test', 30)

for name, X, y in [('train', Xtr, ytr), ('val', Xva, yva), ('test', Xte, yte)]:
    print(name, X.shape, {NAMES[c]: int((y == c).sum()) for c in CLASSES})
```

OUTPUT

```
train (400, 128, 128) {'eosinophil': 100, 'monocyte': 100, 'neutrophil': 100, 'platelet':
100}
val (120, 128, 128) {'eosinophil': 30, 'monocyte': 30, 'neutrophil': 30, 'platelet': 30}
test (120, 128, 128) {'eosinophil': 30, 'monocyte': 30, 'neutrophil': 30, 'platelet': 30}
```

CODE CELL 4

```
fig, axes = plt.subplots(4, 4, figsize=(8, 8))
for r, c in enumerate(CLASSES):
    idx = np.where(ytr == c)[0][:4]
    for k, i in enumerate(idx):
        axes[r, k].imshow(Xtr[i], cmap='gray'); axes[r, k].axis('off')
        if k == 0: axes[r, k].set_title(NAMES[c], loc='left')
plt.tight_layout(); plt.show()
```

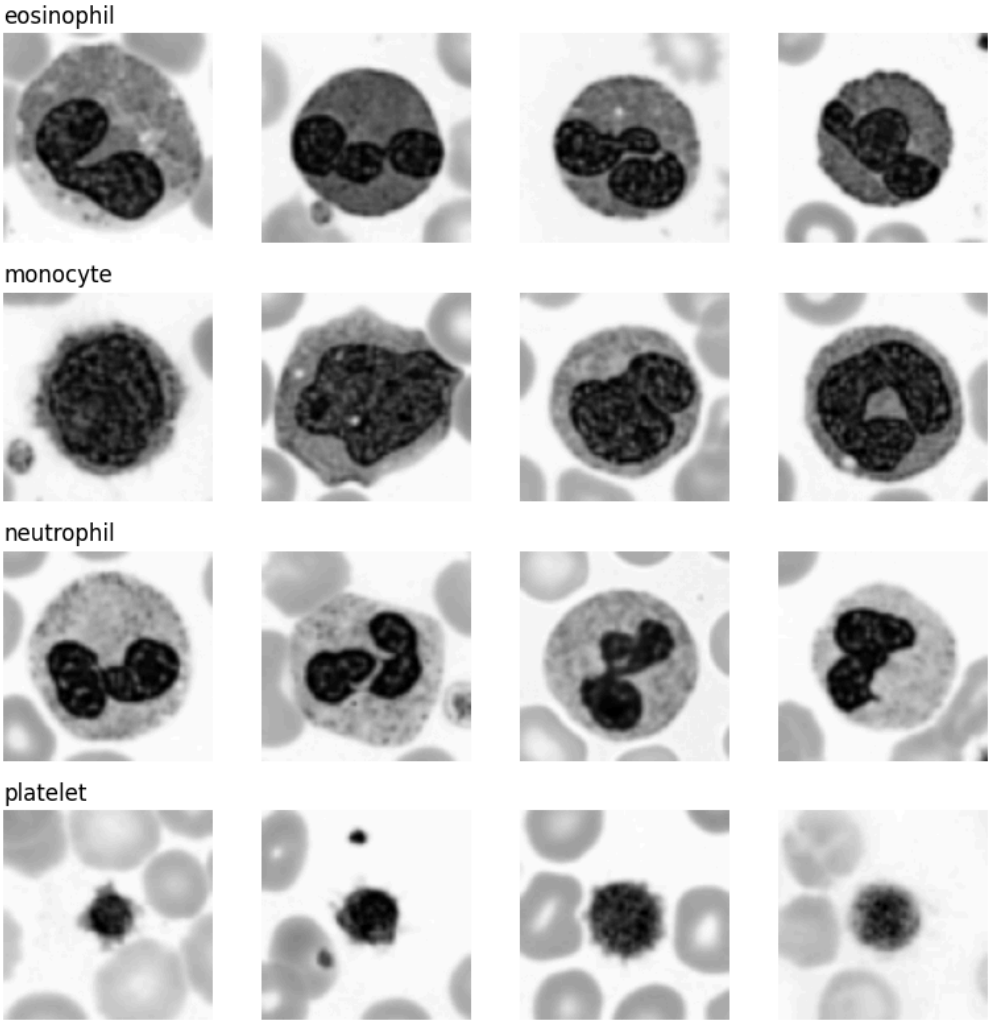


Figure 1 (cell 4)

Q2. HOG + SVM

Hand calculation: 128 / 8 = 16 cells per side. Blocks per side = 16 − 2 + 1 = 15.

Feature length = 15 × 15 blocks × (2 × 2 cells) × 9 orientations = 225 × 36 = 8100.

CODE CELL 6

```
def hog_feats(X):
    return np.array([hog(im, orientations=9, pixels_per_cell=HOG_CELL,
                         cells_per_block=(2, 2), feature_vector=True) for im in X])

Ftr, Fva, Fte = hog_feats(Xtr), hog_feats(Xva), hog_feats(Xte)
print('HOG feature length:', Ftr.shape[1])
```

OUTPUT

```
HOG feature length: 8100
```

CODE CELL 7

```
fig, axes = plt.subplots(2, 4, figsize=(10, 5))
for k, c in enumerate(CLASSES):
    i = np.where(ytr == c)[0][0]
    _, vis = hog(Xtr[i], orientations=9, pixels_per_cell=HOG_CELL,
                 cells_per_block=(2, 2), visualize=True)
    axes[0, k].imshow(Xtr[i], cmap='gray'); axes[0, k].set_title(NAMES[c]); axes[0,
k].axis('off')
    axes[1, k].imshow(vis, cmap='gray'); axes[1, k].axis('off')
plt.show()
```

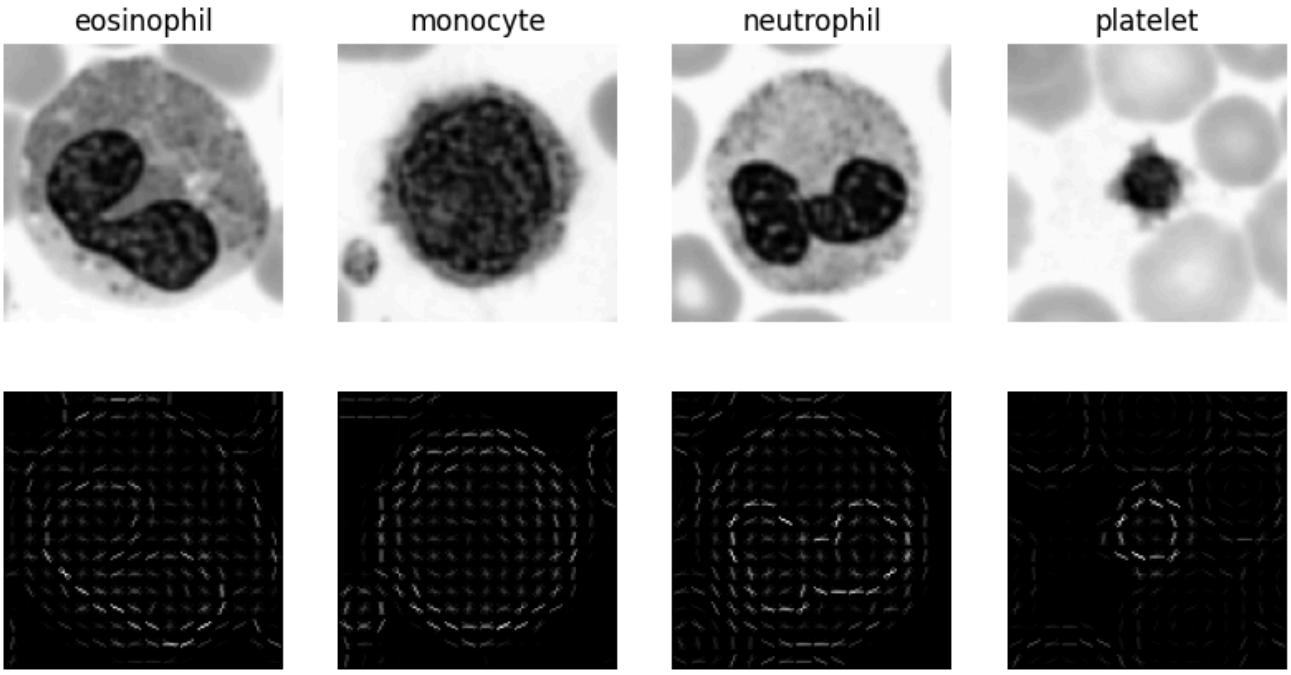


Figure 2 (cell 7)

CODE CELL 8

```
def pick_svm(Ftr, ytr, Fva, yva):
    best = None
    for C in [0.1, 1, 10]:
        clf = LinearSVC(C=C, max_iter=10000).fit(Ftr, ytr)
        acc = accuracy_score(yva, clf.predict(Fva))
        print(f'C={C}: val acc={acc:.3f}')
        if best is None or acc > best[0]:
            best = (acc, C, clf)
    return best

hog_acc, hog_C, hog_clf = pick_svm(Ftr, ytr, Fva, yva)
pred = hog_clf.predict(Fva)
print('Best C:', hog_C)
print('Val accuracy:', accuracy_score(yva, pred))
print('Macro-F1:', f1_score(yva, pred, average='macro'))
print(confusion_matrix(yva, pred, labels=CLASSES))
```

OUTPUT

```
C=0.1: val acc=0.708
C=1: val acc=0.725
C=10: val acc=0.725
Best C: 1
Val accuracy: 0.725
Macro-F1: 0.7232825917252146
[[18  7  5  0]
 [ 5 21  3  1]
 [ 7  5 18  0]
 [ 0  0  0 30]]
```

Q3. SIFT + SVM.

CODE CELL 10

```
sift = cv2.SIFT_create()
counts = []
for im in Xtr:
    kps, _ = sift.detectAndCompute(im, None)
    counts.append(len(kps))
counts = np.array(counts)
print('Keypoints: mean %.1f, min %d, max %d' % (counts.mean(), counts.min(),
counts.max()))
print('Images with zero keypoints:', int((counts == 0).sum()))

kps, _ = sift.detectAndCompute(Xtr[0], None)
vis = cv2.drawKeypoints(Xtr[0], kps, None,
flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
plt.imshow(vis, cmap='gray'); plt.axis('off'); plt.show()
```

OUTPUT

```
Keypoints: mean 70.4, min 6, max 213
Images with zero keypoints: 0
```

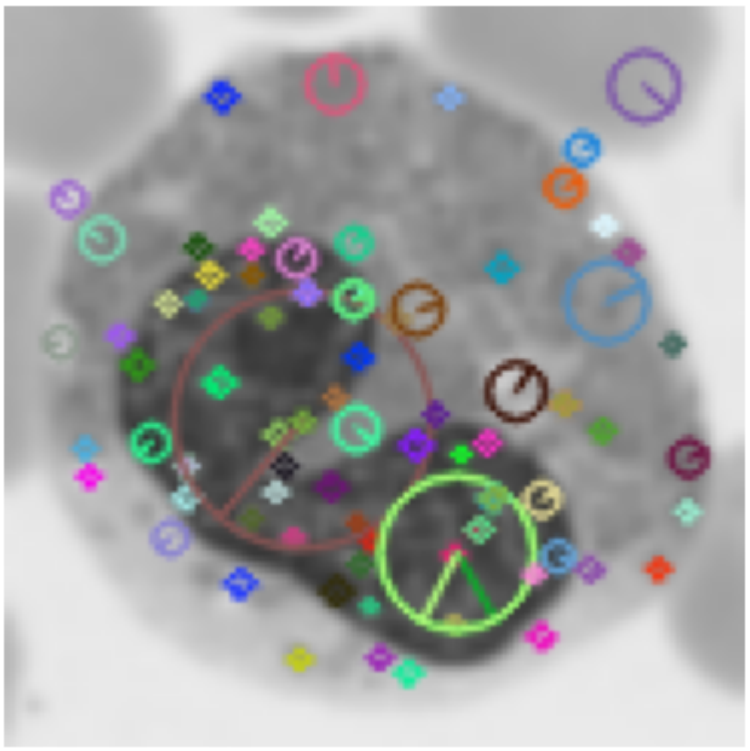


Figure 3 (cell 10)

CODE CELL 11

```
def grid_kps(n=SIFT_GRID, size=32):
    step = 128 / n
    return [cv2.KeyPoint(x=step / 2 + i * step, y=step / 2 + j * step, size=size)
            for j in range(n) for i in range(n)]

def sift_grid_feats(X):
    out = []
    for im in X:
        kps, desc = sift.compute(im, grid_kps())
        assert desc is not None and desc.shape[0] == SIFT_GRID ** 2, 'some grid points
were dropped'
        out.append(desc.flatten())
    return np.array(out)

Str, Sva, Ste = sift_grid_feats(Xtr), sift_grid_feats(Xva), sift_grid_feats(Xte)
print('SIFT grid feature length:', Str.shape[1])
```

OUTPUT

```
SIFT grid feature length: 2048
```

CODE CELL 12

```
sift_acc, sift_C, sift_clf = pick_svm(Str, ytr, Sva, yva)
pred = sift_clf.predict(Sva)
print('Best C:', sift_C)
print('Val accuracy:', accuracy_score(yva, pred))
print('Macro-F1:', f1_score(yva, pred, average='macro'))
print(confusion_matrix(yva, pred, labels=CLASSES))
```

OUTPUT

```
/usr/local/lib/python3.13/dist-packages/sklearn/svm/_base.py:1249: ConvergenceWarning:
Liblinear failed to converge, increase the number of iterations.
  warnings.warn(
```

OUTPUT

```
C=0.1: val acc=0.742
```

OUTPUT

```
/usr/local/lib/python3.13/dist-packages/sklearn/svm/_base.py:1249: ConvergenceWarning:
Liblinear failed to converge, increase the number of iterations.
  warnings.warn(
```

OUTPUT

```
C=1: val acc=0.733
C=10: val acc=0.733
Best C: 0.1
Val accuracy: 0.7416666666666667
Macro-F1: 0.7426372015081693
[[18  9  3  0]
 [ 9 21  0  0]
 [ 6  0 21  3]
 [ 0  0  1 29]]
```

OUTPUT

```
/usr/local/lib/python3.13/dist-packages/sklearn/svm/_base.py:1249: ConvergenceWarning:
Liblinear failed to converge, increase the number of iterations.
  warnings.warn(
```

Q4. Robustness

Prediction (write before running): [your prediction for rotation and noise, with one reason each]

CODE CELL 14

```
np.random.seed(SEED)

def rotate_imgs(X):
    return np.array([rotate(im, 20, preserve_range=True).astype(np.uint8) for im in X])

def noisy_imgs(X, sigma=0.05):
    out = []
    for im in X:
        f = im / 255.0 + np.random.normal(0, sigma, im.shape)
        out.append((np.clip(f, 0, 1) * 255).astype(np.uint8))
    return np.array(out)

cases = {'clean': Xte, 'rotated': rotate_imgs(Xte), 'noisy': noisy_imgs(Xte)}
```

CODE CELL 15

```
rows = {}
for case, X in cases.items():
    hog_pred  = hog_clf.predict(hog_feats(X))
```

```
    sift_pred = sift_clf.predict(sift_grid_feats(X))
    rows[case] = (accuracy_score(yte, hog_pred), accuracy_score(yte, sift_pred))

print('Model        | Clean  | Rotated | Noisy')
print('HOG + SVM    | %.3f  | %.3f   | %.3f' % (rows['clean'][0], rows['rotated'][0],
rows['noisy'][0]))
print('SIFT + SVM   | %.3f  | %.3f   | %.3f' % (rows['clean'][1], rows['rotated'][1],
rows['noisy'][1]))
```

OUTPUT

```
Model        | Clean  | Rotated | Noisy
HOG + SVM    | 0.750  | 0.675   | 0.492
SIFT + SVM   | 0.792  | 0.500   | 0.783
```

Q5. Short answers and AI log

(a) Why HOG uses block normalisation:

Gradient strength changes with lighting and local contrast, so the same object can give very different histogram values. Normalising each block of cells makes the descriptor depend on the pattern of edges rather than their absolute brightness. Because blocks overlap, each cell is normalised several times, which adds further robustness.

(b) Why choosing C on the test set is wrong:

The test set must stay unseen so it estimates performance on new data. If C is chosen by comparing test scores, the value that happens to fit that particular test set is selected, so the reported accuracy is optimistically biased. C should be tuned on a validation split, and the test set used only once at the end.

Most confused classes: eosinophil and monocyte (7 eosinophils predicted as monocyte, 5 monocytes predicted as eosinophil, 12 mixed errors in HOG validation). Eosinophil and neutrophil have the same total (12).

Why grayscale might make this harder: in grayscale, eosinophils and monocytes both appear as dark, dense cells. The pink-orange granule colour of eosinophils and the lighter, kidney-shaped nucleus of monocytes are lost, leaving shape and texture cues that look similar.

AI log

```
| Prompt (one line) | What I changed | How I checked the result was correct |
|---|---|---|
| "Write Python code using HOG (8x8 cells, 2x2 blocks) and LinearSVC to classify 4 blood cell classes from 128x128 grayscale BloodMNIST images" | Kept the 8x8 cell size, tuned C over [0.1, 1, 10] on the validation set, and used the test set only once at the end | Printed the HOG feature length (8100, matching the hand calculation of 15 × 15 × 36) and checked the HOG visualisations for each class |
```